

```

// secure_monitor.c
// "TrustZone-ish" minimal skeleton for RISC-V
// - M-mode acts as Secure Monitor
// - PMP protects secure RAM (keys, cert store)
// - Normal world calls into monitor via ecall (trap)
// NOTE: This is a teaching skeleton; you need startup/linker/CSR helpers for real build.

#include <stdint.h>
#include <stddef.h>

#define SECURE_BASE 0x80200000UL
#define SECURE_SIZE 0x00020000UL // 128KB secure region (example)
#define NORMAL_BASE 0x80000000UL
#define NORMAL_SIZE 0x00200000UL

// Simple "service IDs" for ecall
enum {
    SVC_GET_VERSION = 0x01,
    SVC_SIGN_SHA256 = 0x02,
    SVC_VERIFY_X509_SUBJECT = 0x03,
};

// --- CSR helpers (very minimal) ---
static inline uint64_t read_csr(const char* csr_name) { (void)csr_name; return 0; }
static inline void write_csr(const char* csr_name, uint64_t v) { (void)csr_name; (void)v; }

// You'd normally implement with asm:
// csrr / csrw for mtvec, mepc, mcause, mtval, mstatus, pmpcfg*, pmpaddr*

// --- PMP config encoding notes ---
// RISC-V PMP uses pmpcfgX and pmpaddrX.
// Common approach for a "secure region deny" is to allow everything for M-mode,
// but deny S/U access to SECURE region.
// For teaching: we'll set one PMP entry as "no access for S/U" to secure range,
// and another as "RWX for normal RAM" for S/U.

static void pmp_config_secure_deny(void) {
    // PSEUDO: You must write pmpaddr0 + pmpcfg0 with NAPOT or TOR.
    // We'll describe intent with comments.

    // 1) Entry0: SECURE region = NO access for S/U (R=0,W=0,X=0)
    // 2) Entry1: NORMAL region = RWX for S/U (R=1,W=1,X=1)

    // Real CSR names: pmpaddr0, pmpcfg0, pmpaddr1, pmpcfg0 etc.

    // Example NAPOT encoding:
    // pmpaddr = (base >> 2) | ((size/2 - 1) >> 2) ... (depends on size power-of-two)
    // cfg: A=NAPOT, R/W/X bits, L lock optional

```

```

//
// Here we omit raw encoding and keep it conceptual.
}

typedef struct {
    uint64_t ra;
    uint64_t sp;
    uint64_t gp;
    uint64_t tp;
    uint64_t t0,t1,t2;
    uint64_t s0,s1;
    uint64_t a0,a1,a2,a3,a4,a5,a6,a7;
    uint64_t s2,s3,s4,s5,s6,s7,s8,s9,s10,s11;
    uint64_t t3,t4,t5,t6;
    uint64_t mepc;
    uint64_t mcause;
    uint64_t mtval;
} trap_frame_t;

// --- "secure storage" (would be in secure RAM) ---
static uint8_t g_device_private_key[32] = { /* demo only */ 1,2,3 };
static uint32_t g_version = 0x00010001;

// VERY IMPORTANT: in real systems you never expose key material; only operations.
static int secure_sign_sha256(const uint8_t hash[32], uint8_t sig_out[64]) {
    // Demo "signature": NOT REAL CRYPTO. Replace with real ECC (P-256/Ed25519)
    // implementation.
    // We'll just mix bytes to show data flow.
    for (int i=0;i<32;i++) {
        sig_out[i] = (uint8_t)(hash[i] ^ g_device_private_key[i % 32]);
        sig_out[i+32] = (uint8_t)(hash[i] + g_device_private_key[i % 32]);
    }
    return 0;
}

static int secure_verify_x509_subject(const uint8_t* cert_der, size_t cert_len) {
    // Demo stub: real X.509 parsing is non-trivial.
    // Here you'd parse DER, check validity window, EKU, chain (or pinned CA),
    // then decide "subject allowed?"
    (void)cert_der; (void)cert_len;
    return 1; // "allowed" (demo)
}

// --- Trap handler (M-mode) ---
void m_trap_handler(trap_frame_t* tf) {
    // In real code, read mcause/mepc/mtval from CSRs.
    // Here assume tf contains them.
    uint64_t cause = tf->mcause;

```

```

// On RISC-V, ECALL from U-mode is typically cause == 8 (Environment call from U-mode)
// ECALL from S-mode is cause == 9.
const uint64_t CAUSE_ECALL_U = 8;
const uint64_t CAUSE_ECALL_S = 9;

if (cause == CAUSE_ECALL_U || cause == CAUSE_ECALL_S) {
    uint64_t svc = tf->a7; // convention: a7 holds syscall/service id
    switch (svc) {
        case SVC_GET_VERSION:
            tf->a0 = g_version;
            break;

        case SVC_SIGN_SHA256: {
            // a0 = ptr to 32-byte hash in NORMAL RAM
            // a1 = ptr to 64-byte sig_out in NORMAL RAM
            const uint8_t* hash = (const uint8_t*)(uintptr_t)tf->a0;
            uint8_t* sig_out = (uint8_t*)(uintptr_t)tf->a1;

            // Here you SHOULD validate pointers are in NORMAL region, not SECURE region.
            // For demo, assume valid.
            int rc = secure_sign_sha256(hash, sig_out);
            tf->a0 = (uint64_t)rc;
            break;
        }

        case SVC_VERIFY_X509_SUBJECT: {
            // a0 = cert ptr, a1 = cert len
            const uint8_t* cert = (const uint8_t*)(uintptr_t)tf->a0;
            size_t len = (size_t)tf->a1;

            int ok = secure_verify_x509_subject(cert, len);
            tf->a0 = (uint64_t)ok;
            break;
        }

        default:
            tf->a0 = (uint64_t)-1;
            break;
    }

    // Advance mepc to skip the ecall instruction (4 bytes for RV32/64 base ISA)
    tf->mepc += 4;
    // In real code: write mepc CSR back.
    return;
}

// Other traps: illegal instr, access fault, etc.

```

```

// In real system you'd log + reset or kill offending task.
while (1) { /* halt */ }
}

void monitor_init(void) {
    // 1) Configure PMP so Normal world cannot read SECURE region.
    pmp_config_secure_deny();

    // 2) Set mtvec to trap entry (needs asm wrapper that saves regs into trap_frame_t)
    // write_csr("mtvec", (uint64_t)&m_trap_entry);

    // 3) Drop into Normal world entrypoint (U/S-mode) by setting mstatus/mepc then mret.
    // This part is platform/toolchain specific.
}

```